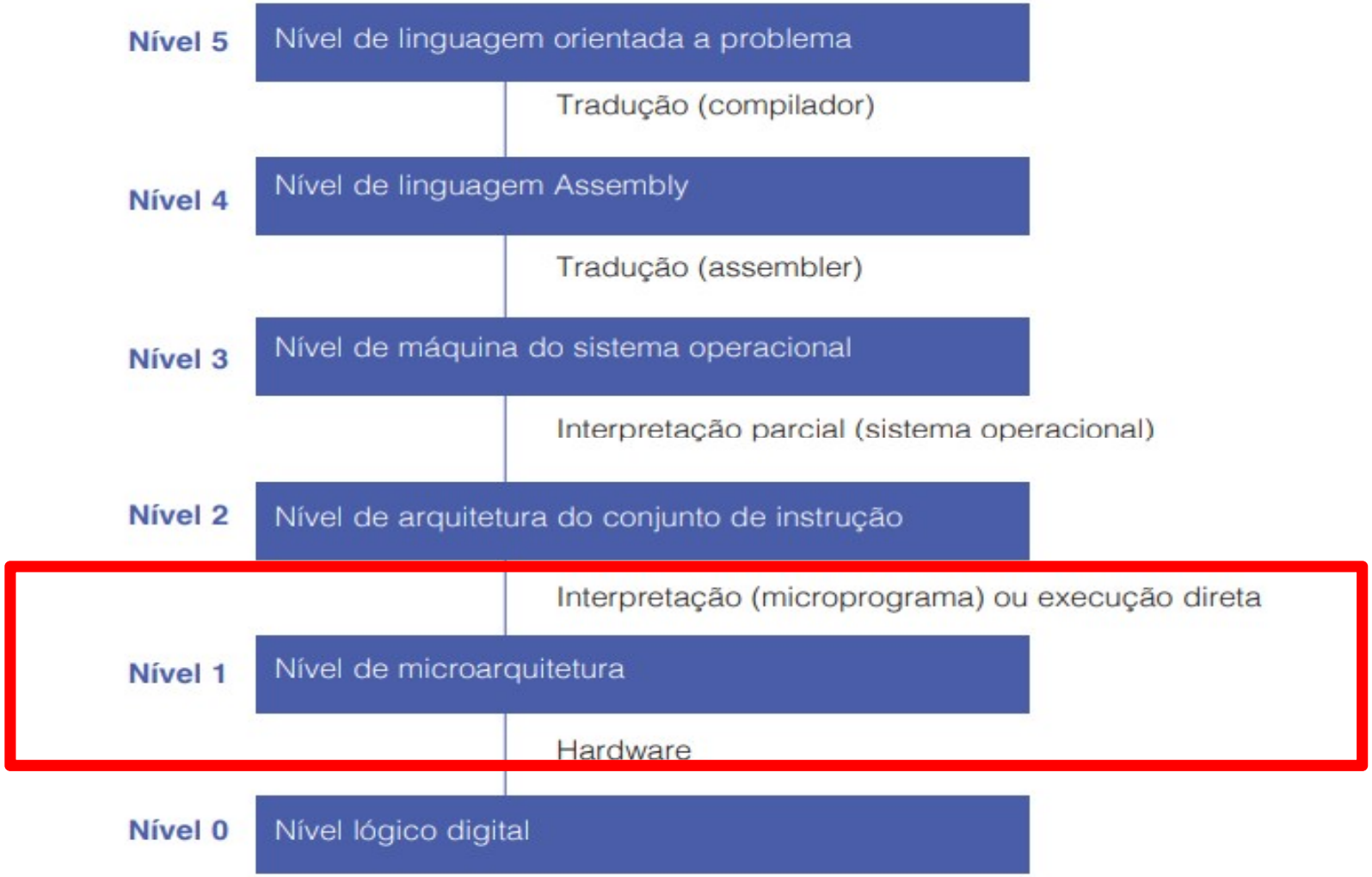
Nível de microarquitetura

Controle de fluxo

Sequência de instruções.

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



Pilhas

- Praticamente todas as linguagens de programação trabalham com o conceito de procedimentos (ou métodos), que têm **variáveis locais.**
  - Essas variáveis podem ser acessadas de dentro dos procedimentos, mas deixam de ser acessíveis assim que o procedimento é devolvido.
- **Pergunta: Em que lugar da memória essas variáveis devem ser mantidas?**

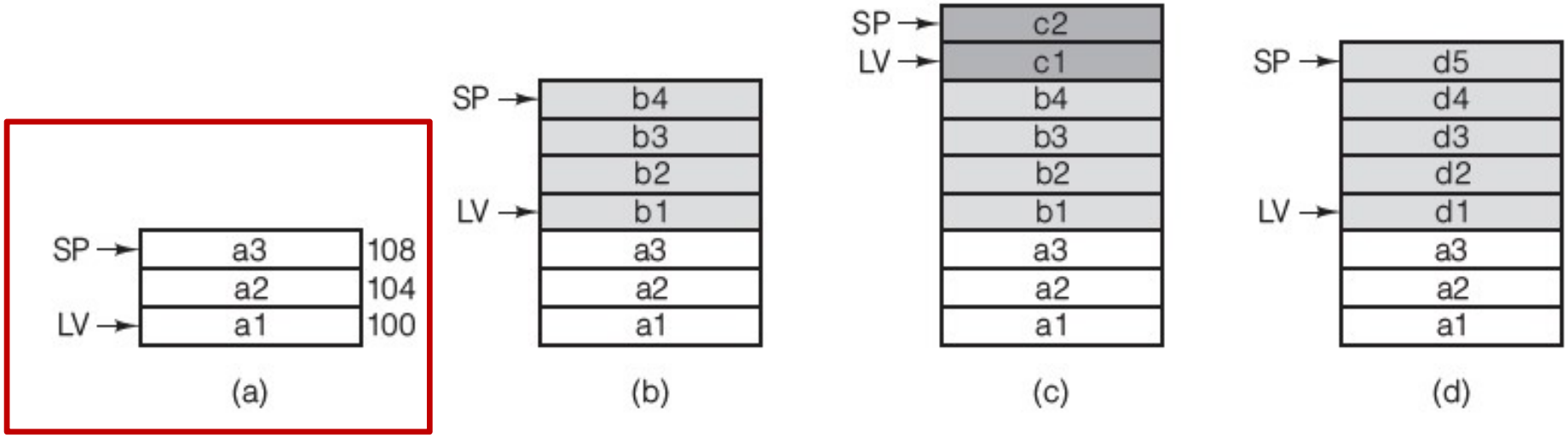
Pilhas

- A solução mais simples, dar a cada variável um endereço de memória absoluto, não funciona.
- O problema é que um procedimento pode chamar a si mesmo.
  - Se um procedimento for ativado – isto é, chamado – duas vezes, é impossível armazenar suas variáveis em localizações absolutas de memória porque a segunda chamada irá interferir com a primeira.
- Em vez disso, é usada uma estratégia diferente. Uma área da memória, denominada **pilha**, é reservada para variáveis locais.
- Um registrador chamado **LV é preparado para apontar para a base das variáveis locais** para o procedimento em questão.

Pilhas

- Na Figura, um procedimento A, que tem variáveis locais a1, a2 e a3, foi chamado
  - foi reservado armazenamento para suas variáveis locais, **começando na localização de memória apontada por LV.**
- **Outro registrador, SP, aponta para a palavra mais alta das variáveis locais de A.** Se LV for 100 e as palavras tiverem 4 bytes, então SP será 108.

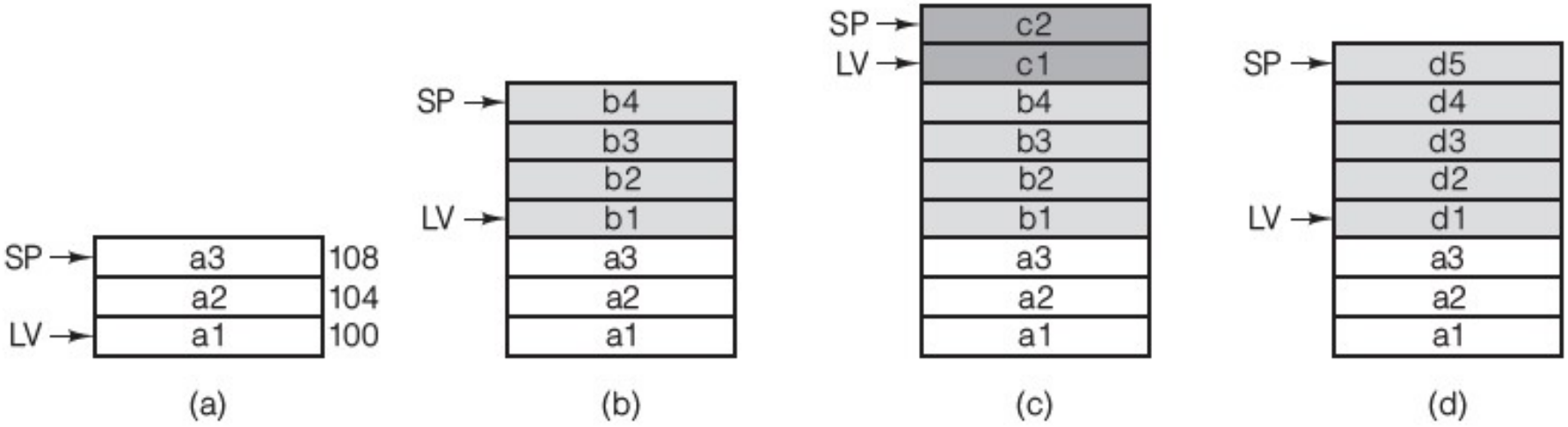
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- A estrutura de dados entre LV e SP (e incluindo ambas as palavras apontadas) é denominada **quadro de variáveis locais de A.**

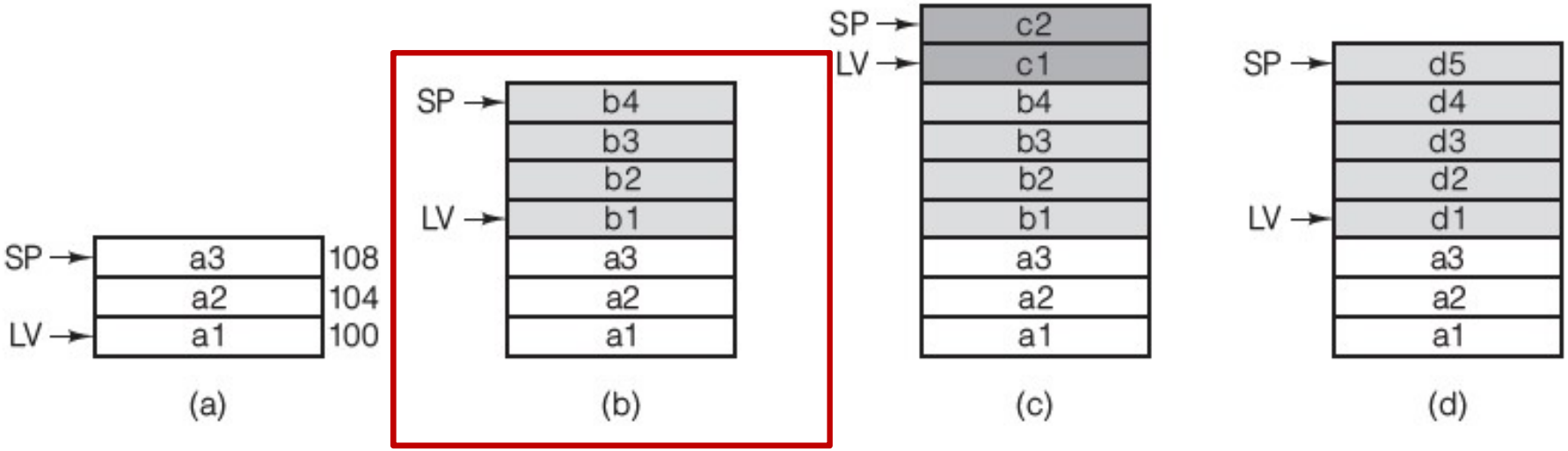
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- Considere: A chamar outro método B.
- Onde deveriam ser armazenadas as quatro variáveis locais de B (b1, b2, b3, b4)?

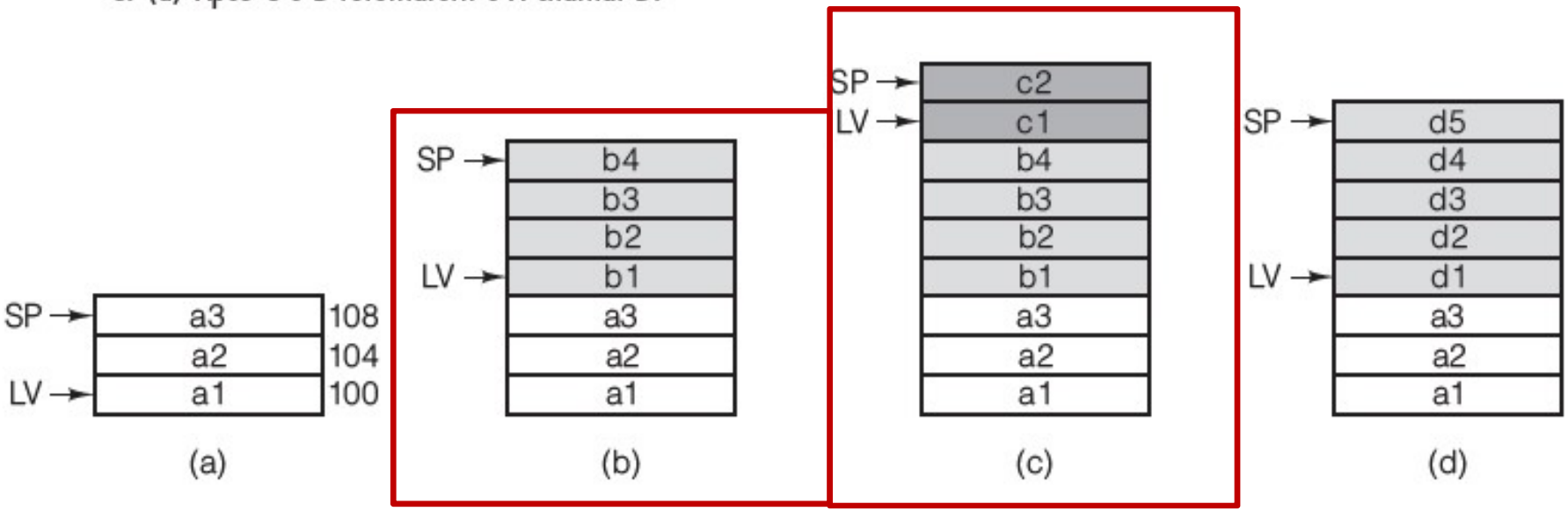
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- Resposta: na pilha, em cima das variáveis de A.
- Observe que LV foi ajustado pela chamada de procedimento para que aponte para as variáveis locais de B em vez das de A.
- As variáveis locais de B podem ser referenciadas dando seu deslocamento em relação a LV.
- De modo semelhante, se B chamar C, LV e SP são ajustados novamente para alocar espaço para as duas variáveis de C, como mostra a Figura 4.8(c).

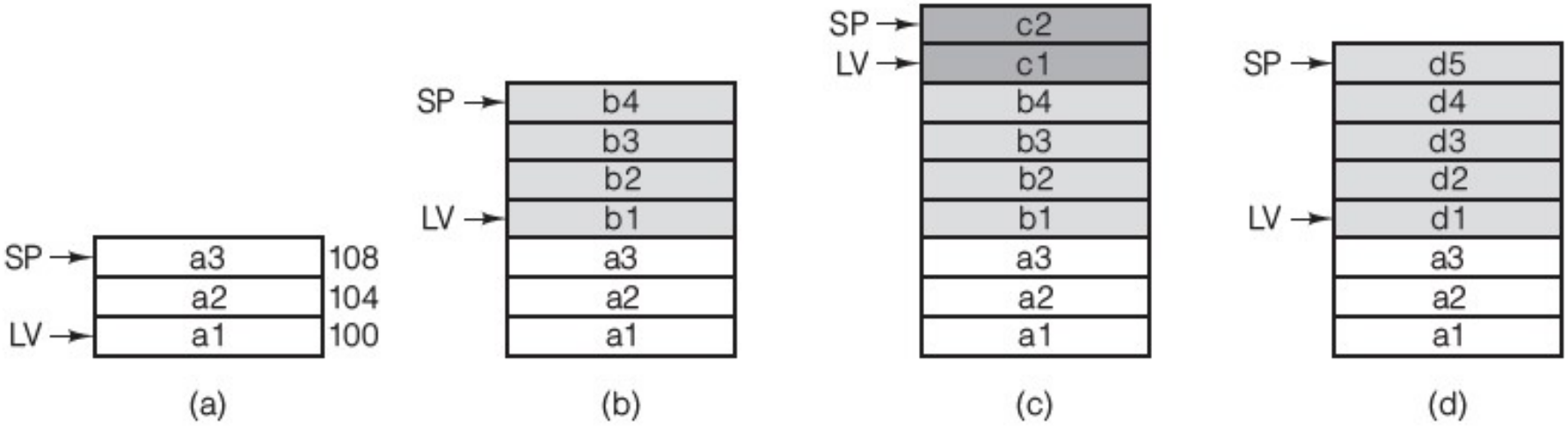
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- Quando C retorna, B torna-se ativo de novo e a pilha volta a ser ajustada para a Figura 4.8(b), LV aponta outra vez para as variáveis locais de B.
- Da mesma forma, quando B retorna, voltamos à situação da Figura 4.8(a).
  - Sob todas as condições, LV aponta para a base do quadro da pilha para o procedimento ativo no momento em questão e SP aponta para o topo do quadro da pilha.

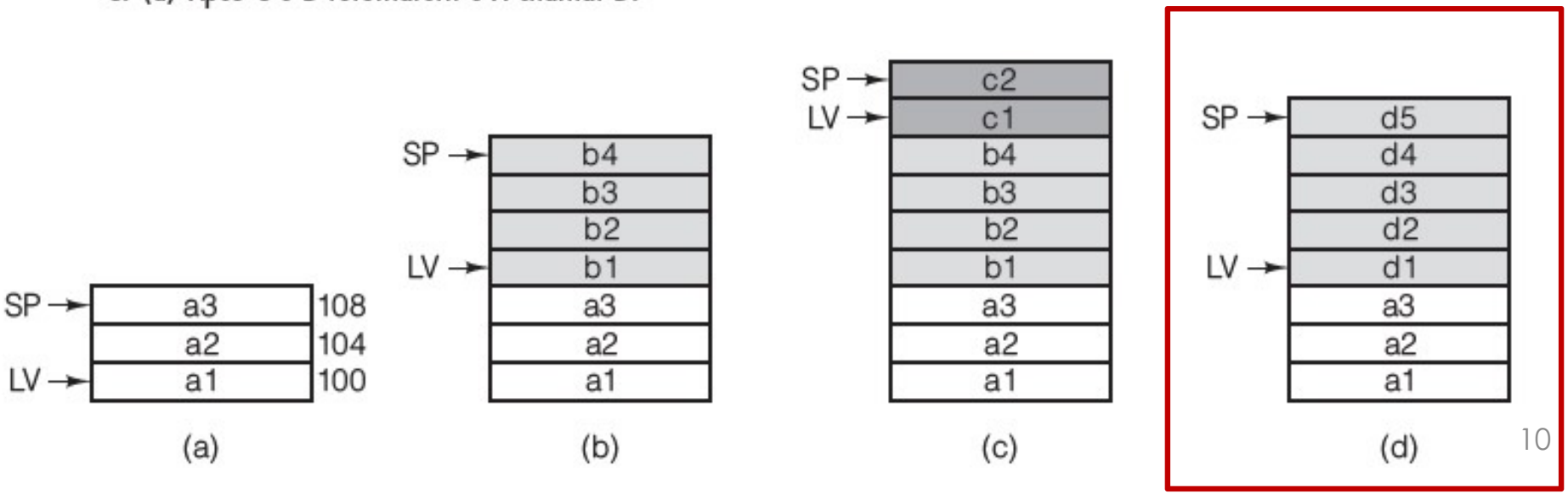
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- Suponha que A chama D, que tem cinco variáveis locais.
- Essa é a situação da Figura 4.8(d), na qual as variáveis locais de D usam a mesma memória que as de B usaram, bem como parte das de C.
- Com essa organização, a memória só é alocada para procedimentos que estão ativos no momento.
- Quando um método retorna, a memória usada por suas variáveis locais é liberada.

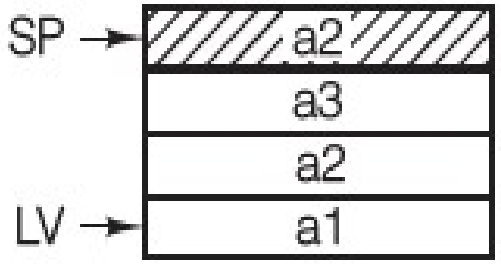
Figura 4.8 Utilização de uma pilha para armazenar variáveis locais. (a) Enquanto *A* está ativo. (b) Após *A* chamar *B*. (c) Após *B* chamar *C*. (d) Após *C* e *B* retornarem e *A* chamar *D*.



Pilhas

- Pilhas têm outra utilização além de conter variáveis locais.
  – Elas podem ser usadas para **reter operandos** durante o cálculo de uma expressão aritmética.
- Quando usada dessa maneira, a pilha é denominada **pilha de operandos.**
- Suponha, por exemplo, que, antes de chamar B, A tenha de calcular

  a1 = a2 + a3;
- Um modo de efetuar essa soma é passar a2 para a pilha, como ilustra a Figura 4.9(a).
- Nesse caso, SP foi incrementado pelo número de bytes em uma palavra, por exemplo, 4, e o primeiro operando foi armazenado no endereço agora apontado por SP.

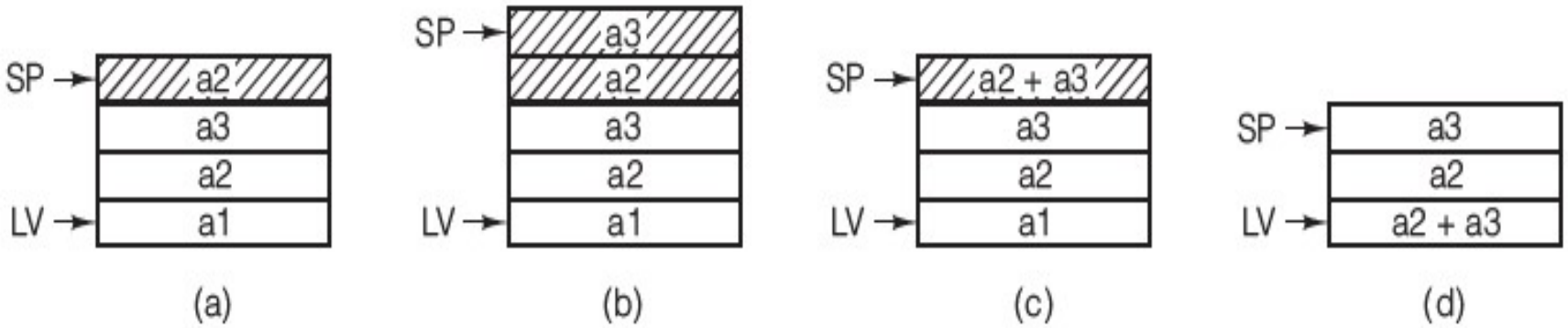


(a)

Pilhas

- Em seguida, a3 é passada para a pilha, conforme mostra a Figura 4.9(b).
- Agora, o cálculo pode ser feito executando uma instrução que retira duas palavras da pilha, soma as duas e devolve o resultado para a pilha, conforme a Figura 4.9(c).
- Por fim, a palavra que está no topo pode ser retirada da pilha e armazenada de novo na variável local a1, como ilustrado na Figura 4.9(d).

Figura 4.9 Utilização de uma pilha de operandos para efetuar um cálculo aritmético.



Modelo de memória IJVM

- Estamos prontos para estudar a arquitetura ISA da IJVM.
- Em essência, ela consiste em uma memória que pode ser vista de dois modos: um arranjo de 4.294.967.296 bytes (4 GB) ou um arranjo de 1.073.741.824 palavras, cada uma consistindo em 4 bytes.
- Instruções IJVM só podem acessar a memória a partir de ponteiros.

Modelo de memória IJVM

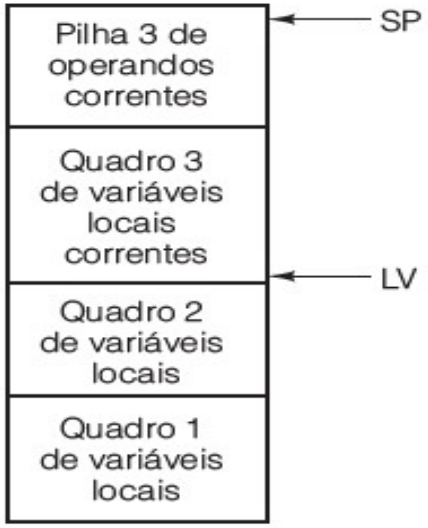
- Em qualquer instante, as seguintes áreas de memória são definidas:

1. O conjunto de constantes. Essa área não pode ser escrita por um programa IJVM e consiste em constantes, cadeias e ponteiros para outras áreas da memória que podem ser referenciadas.
   - Ele é carregado quando o programa é trazido para a memória e não é alterado depois.
   - Há um registrador implícito, CPP, que contém o endereço da primeira palavra do conjunto de constantes.

Modelo de memória IJVM

2. O quadro de variáveis locais. Para cada invocação de um método é alocada uma área para armazenar variáveis durante o tempo de vida da invocação, denominada **quadro de variáveis locais.**

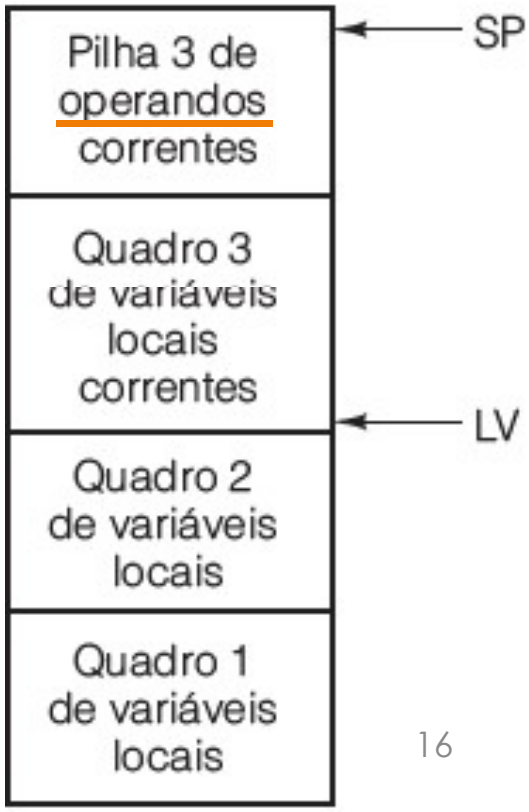
- No início desse quadro estão os parâmetros (também denominados argumentos) com os quais o método foi invocado.
- Há um registrador que contém o endereço da primeira localização do quadro de variáveis locais chamado LV.
- Os parâmetros passados na chamada do método são armazenados no início do quadro de variáveis locais.



Modelo de memória IJVM

3. A pilha de operandos.

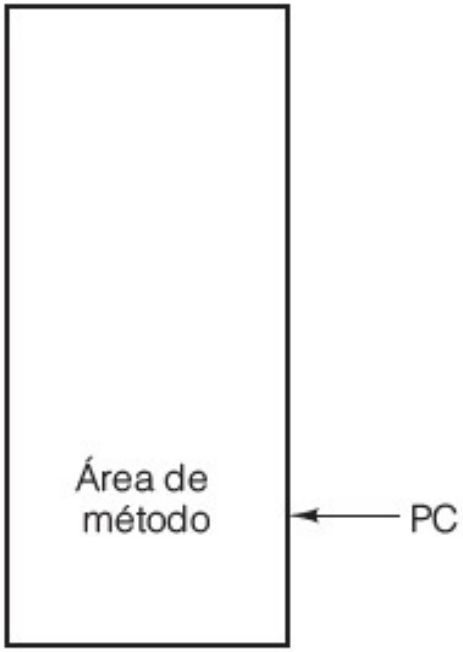
- É garantido que o quadro não exceda certo tamanho, calculado com antecedência.
- O espaço da pilha de operandos é alocado diretamente acima do quadro de variáveis locais.
- É conveniente imaginar a pilha de operandos como parte do quadro de variáveis locais.
- De qualquer modo, há um registrador SP que contém o endereço da palavra do topo da pilha.
- Note que, diferente do CPP e do LV, esse ponteiro, SP, muda durante a execução do método à medida que operandos são passados para a pilha ou retirados dela.



Modelo de memória IJVM

4. A área de método.

- Há uma região da memória que contém o programa.
- Há um registrador que contém o endereço da instrução a ser buscada em seguida.
- Esse ponteiro é denominado contador de programa (Program Counter) ou PC.
- Diferente das outras regiões da memória, a área de método é tratada como um vetor de bytes.



Modelo de memória IJVM

- É preciso esclarecer uma questão em relação aos ponteiros. Os registradores CPP, LV e SP são todos ponteiros para palavras, não para bytes, e são deslocados pelo número de palavras.
- Para os inteiros que escolhemos, todas as referências a itens no conjunto de constantes, o quadro de variáveis locais e as pilhas são palavras, e todos os deslocamentos usados para indexar esses quadros são deslocamentos de palavras.

Modelo de memória IJVM

- Por exemplo, LV, LV + 1 e LV + 2 se referem às primeiras três palavras do quadro de variáveis locais.
- Ao contrário, PC contém um endereço de byte, e uma adição ou subtração ao PC altera o endereço por um número de bytes, e não por um número de palavras.
- Incrementar o PC por um fator de um e iniciar uma leitura resulta em uma busca pelo próximo byte.
- Incrementar o SP por um fator de um e iniciar uma leitura resulta em uma busca pela próxima palavra.

Conjunto de instruções da IJVM

- O conjunto de instruções da IJVM é mostrado na Figura 4.11.
- Cada instrução consiste em um *opcode e às* vezes um operando, tal como um deslocamento de memória ou uma constante.
- A primeira coluna dá a codificação hexadecimal da instrução.
- A segunda dá seu mnemônico em linguagem de montagem.
- A terceira dá uma breve descrição de seu efeito.

Figura 4.11 Conjunto de instruções da IJVM. Os operandos *byte*, *const* e *varnum* são de 1 byte. Os operandos *disp*, *index* e *offset* são de 2 bytes.

Hexa	Mnemônico	Significado
0x10	BIPUSH *byte*	Carregue o byte para a pilha
0x59	DUP	Copie a palavra do topo da pilha e passe-a para a pilha
0xA7	GOTO *offset*	Desvio incondicional
0x60	IADD	Retire duas palavras da pilha; carregue sua soma
0x7E	IAND	Retire duas palavras da pilha; carregue AND booleano
0x99	IFEQ *offset*	Retire palavra da pilha e desvie se for zero
0x9B	IFLT *offset*	Retire palavra da pilha e desvie se for menor do que zero
0x9F	IF_ICMPEQ *offset*	Retire duas palavras da pilha; desvie se iguais
0x84	IINC *varnum const*	Some uma constante a uma variável local

0x84	IINC *varnum const*	Some uma constante a uma variável local
0x15	ILOAD *varnum*	Carregue variável local para pilha
0xB6	INVOKEVIRTUAL *disp*	Invoque um método
0x80	IOR	Retire duas palavras da pilha; carregue OR booleano
0xAC	IRETURN	Retorne do método com valor inteiro
0x36	ISTORE *varnum*	Retire palavra da pilha e armazene em variável local
0x64	ISUB	Retire duas palavras da pilha; carregue sua diferença
0x13	LDC_W *index*	Carregue constante do conjunto de constantes para pilha
0x00	NOP	Não faça nada
0x57	POP	Apague palavra no topo da pilha
0x5F	SWAP	Troque as duas palavras do topo da pilha uma pela outra
0xC4	WIDE	Instrução prefixada; instrução seguinte tem um índice de 16 bits

Compilando Java para a IJVM

- Na Figura 4.14:
  - um fragmento simples de código Java.
  - a linguagem de montagem IJVM mostrada na Figura 4.14(b).
  - Então, o *assembler Java traduziria* o programa de montagem para o programa binário mostrado na Figura 4.14(c).
- Para este exemplo, consideremos que *i é a* variável local 1, *j é a variável local* 2 e *k é a variável local* 3.

Compilando Java para a IJVM

Figura 4.14 (a) Fragmento em Java. (b) Linguagem de montagem Java correspondente. (c) Programa IJVM em hexadecimal.

```
i = j + k;
if (i == 3)
    k = 0;
else
   j = j - 1;
```

(a)

```
 1        ILOAD  j          // i = j + k        0x15 0x02
 2        ILOAD  k                              0x15 0x03
 3        IADD                                  0x60
 4        ISTORE i                              0x36 0x01
 5        ILOAD  i          // if (i == 3)      0x15 0x01
 6        BIPUSH 3                              0x10 0x03
 7        IF_ICMPEQ L1                          0x9F 0x00 0x0D
 8        ILOAD j           // j = j - 1        0x15 0x02
 9        BIPUSH 1                              0x10 0x01
10        ISUB                                  0x64
11        ISTORE j                              0x36 0x02
12        GOTO  L2                              0xA7 0x00 0x07
13   L1:  BIPUSH  0         // k = 0            0x10 0x00
14        ISTORE  k                             0x36 0x03
15   L2:
```

(b) (c)

Compilando Java para a IJVM

- Primeiro j e k são passadas para a pilha, somadas e o resultado é armazenado em i.
- Então, i e a constante 3 são passadas para a pilha e comparadas.
- Se forem iguais, é tomado um desvio para L1, onde k é ajustada para 0.
- Se forem diferentes, a comparação falha e o código logo após IF_ICMPEQ é executado.
- Feito isso, ele desvia para L2, onde as partes then e else se fundem.

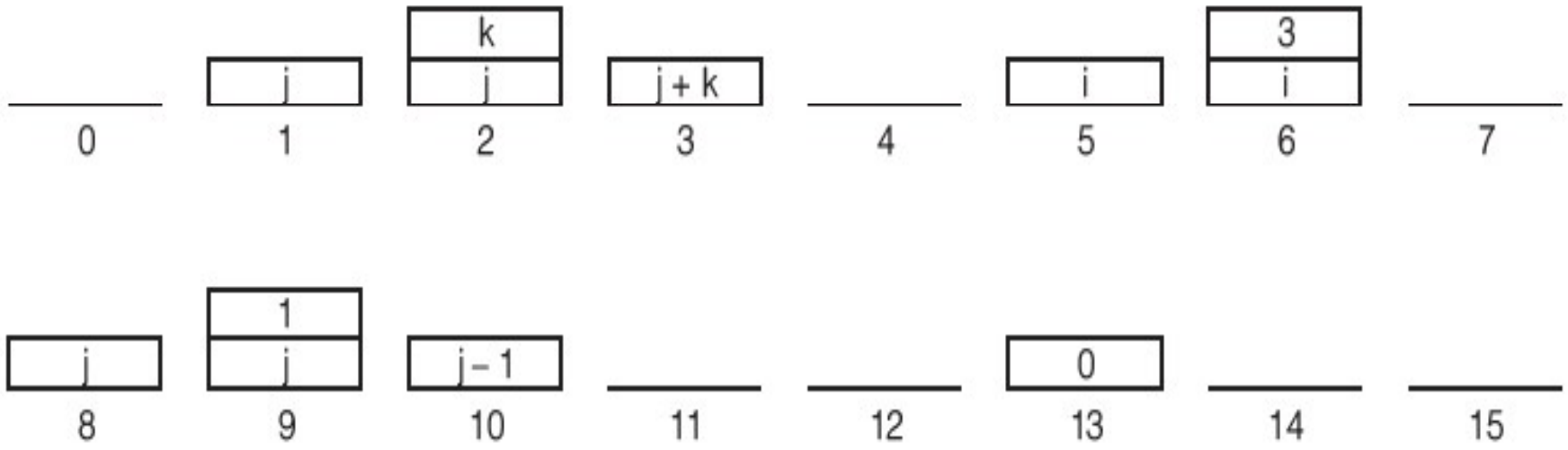
Figura 4.14 (a) Fragmento em Java. (b) Linguagem de montagem Java correspondente. (c) Programa IJVM em hexadecimal.

```
i = j + k;                1     ILOAD j          // i = j + k        0x15 0x02
if (i == 3)               2     ILOAD k                             0x15 0x03
   k = 0;                 3     IADD                                0x60
else                      4     ISTORE i                            0x36 0x01
   j = j - 1;             5     ILOAD i          // if (i == 3)      0x15 0x01
                          6     BIPUSH 3                            0x10 0x03
                          7     IF_ICMPEQ L1                        0x9F 0x00 0x0D
                          8     ILOAD j          // j = j - 1        0x15 0x02
                          9     BIPUSH 1                            0x10 0x01
                         10     ISUB                                0x64
                         11     ISTORE j                            0x36 0x02
                         12     GOTO L2                             0xA7 0x00 0x07
                         13 L1: BIPUSH 0         // k = 0            0x10 0x00
                         14     ISTORE k                            0x36 0x03
                         15 L2:
(a)                       (b)                                       (c)
```

Compilando Java para a IJVM

- A pilha de operandos para o programa IJVM é mostrada na Figura 4.15.
- Antes de o código começar a executar, ela está vazia, o que é indicado pela linha horizontal acima do 0.
- Após a primeira ILOAD, j está na pilha, como indicado por j no retângulo acima de 1.

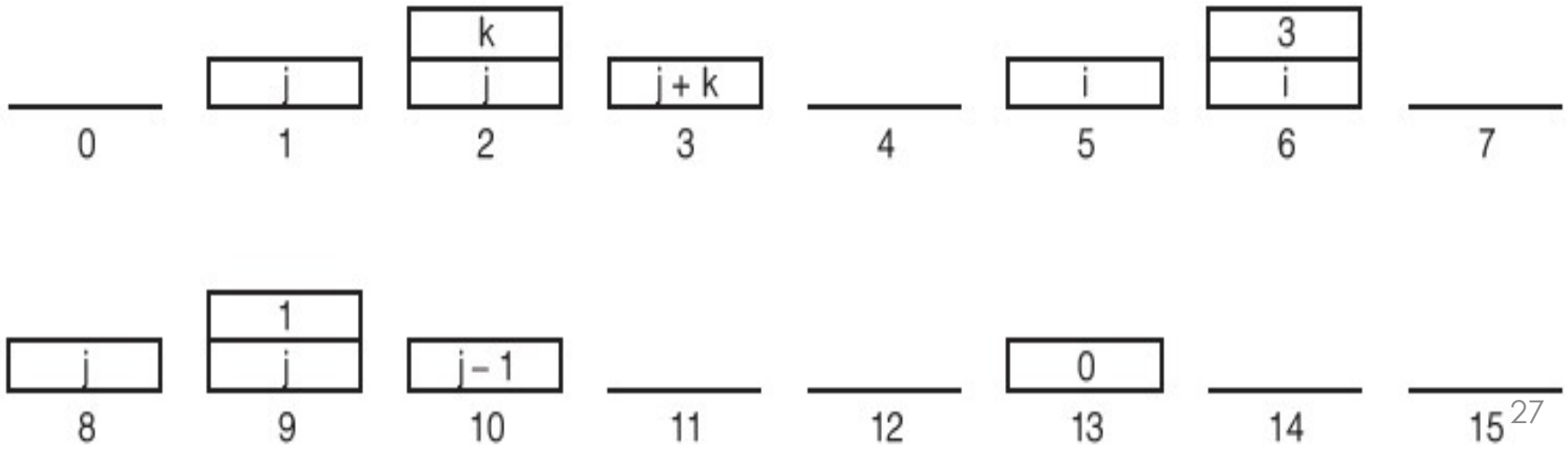
Figura 4.15 Pilha após cada instrução da Figura 4.14(b).



Compilando Java para a IJVM

- Depois da segunda ILOAD, duas palavras estão na pilha, como mostrado acima de 2.
- Após a IADD, há somente uma palavra na pilha, que contém a soma *j + k*.
- *Quando a palavra do topo é retirada e armazenada em i, a pilha* está vazia, como mostrado acima do 4.
- A instrução 5 (ILOAD) inicia a declaração if passando *i para a pilha (em 5)*.
- *Em seguida, vem a constante 3* (em 6).

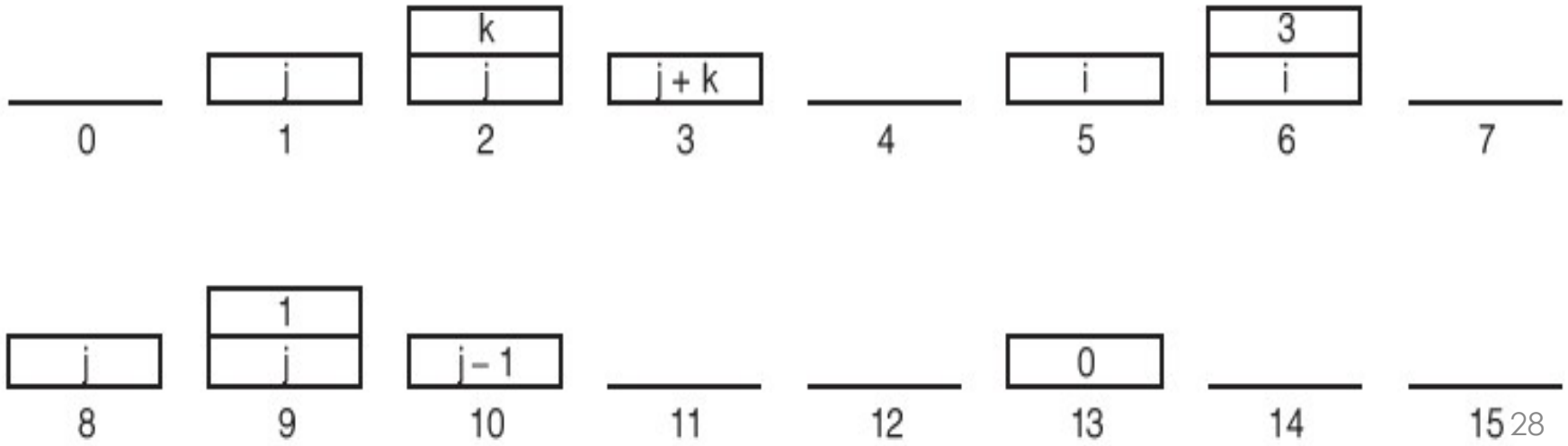
Figura 4.15 Pilha após cada instrução da Figura 4.14(b).



Compilando Java para a IJVM

- Após a comparação, a pilha está novamente vazia (7).
- A instrução 8 é o início da parte else do fragmento de programa Java.
- A parte else continua até a instrução 12, quando então desvia para a parte then e vai para o rótulo *L2*.

Figura 4.15 Pilha após cada instrução da Figura 4.14(b).



Atividade para casa

- Qual é a instrução que invoca o método na microarquitetura (em hexa e em mnemônico)?
- Qual é a instrução que retorna o método na microarquitetura (em hexa e em mnemônico)?
- Como é o comportamento da memória quando há um método para ser executado?
- Como a pilha é montada quando há um método?
- Descreva a função dos registradores vistos nas últimas duas aulas.

Referência

TANENBAUM, Andrew S.; ZUCCHI, Wagner Luiz. **Organização estruturada de computadores**. Pearson Prentice Hall, 2013.